



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Studentų ir įmonių praktikos paieškos platforma

Praktinio darbo ataskaita

Atliko: Rugilė Macaitė, 1 grupė, 1 pogrupis
VU el. p.: rugile.macaite@mif.stud.vu.lt

Vertino: Dr. Andrius Misiukas Misiūnas

TURINYS

Turiny	3
1. Sistemos tikslas	4
2. Numatytas sistemos funkcionalumas	4
2.1. Iteracija	4
2.2. Iteracija	4
2.3. Iteracija	4
2.4. Iteracija	4
3. Naudojamos technologijos	4
4. Nefunkciniai reikalavimai	5
5. Panaudos atviej diagrama	5
6. Reliacinės Duomenų Bazės Schema	6
7. Aktyvumo diagrama	7
8. Pakeitimai	8
8.1. Iteracijų pakeitimas	8
8.2. Technologinių sprendimų pakeitimas	8
8.3. Panaudos atvejų diagramos pakeitimas	8
8.4. Esių ryšių diagramos pakeitimai	9
8.5. Aktyvumo diagramos pakeitimai	10
9. Paleidimo instrukcija	12
9.1. Repositorijos klonavimas	12
9.2. Duomenų bazės sukūrimas	13
9.3. Google OAuth konfigūracija	13
9.4. Environment variables konfigūracija	14
9.5. Dependencies įdiegimas	14
9.6. Paleidimas	14
10. Projekto struktūra	15

1. SISTEMOS TIKSLAS

Kuriama platforma, skirta studentams, ieškantiems praktikos vietų, ir įmonėms, kurios nori rasti praktikantų. Studentai galės susikurti savo profilį, įkelti CV, nurodyti įgūdžius ir ieškoti jiems tinkamų praktikos pasiūlymų. Įmonės galės kurti savo paskyras, skelbti praktikos vietas.

2. NUMATYTAS SISTEMOS FUNKCIONALUMAS

Sistemos kūrimas padalintas į keturias iteracijas, kuriose bus įgyvendintos 2 – 3 užduotys per iteraciją.

2.1. Iteracija

- Prisijungimas/registracija su Google auth
- Profilio sukūrimas (rolės pasirinkimas: studentas ar įmonė, kita vartotojo informacija)

2.2. Iteracija

- CV failo įkėlimo funkcija (per abi roles)
- Funkcija leidžianti įmonėms skelbti praktikos vietas
- Skelbimų paieška naudojant filtrą

2.3. Iteracija

- Skelbimų paieškos laukelis (įvedant informaciją)
- Kandidatavimo į praktiką funkcija

2.4. Iteracija

- Kandidatų sąrašas prie kiekvienos praktikos (mato tik įmonė)
- Priimti arba atmesti kandidatus, atsiunčiamas patvirtinimo laiškas
- Pateiktų savo paraiškų sąrašas (mato ieškantysis praktikos)

3. NAUDOJAMOS TECHNOLOGIJOS

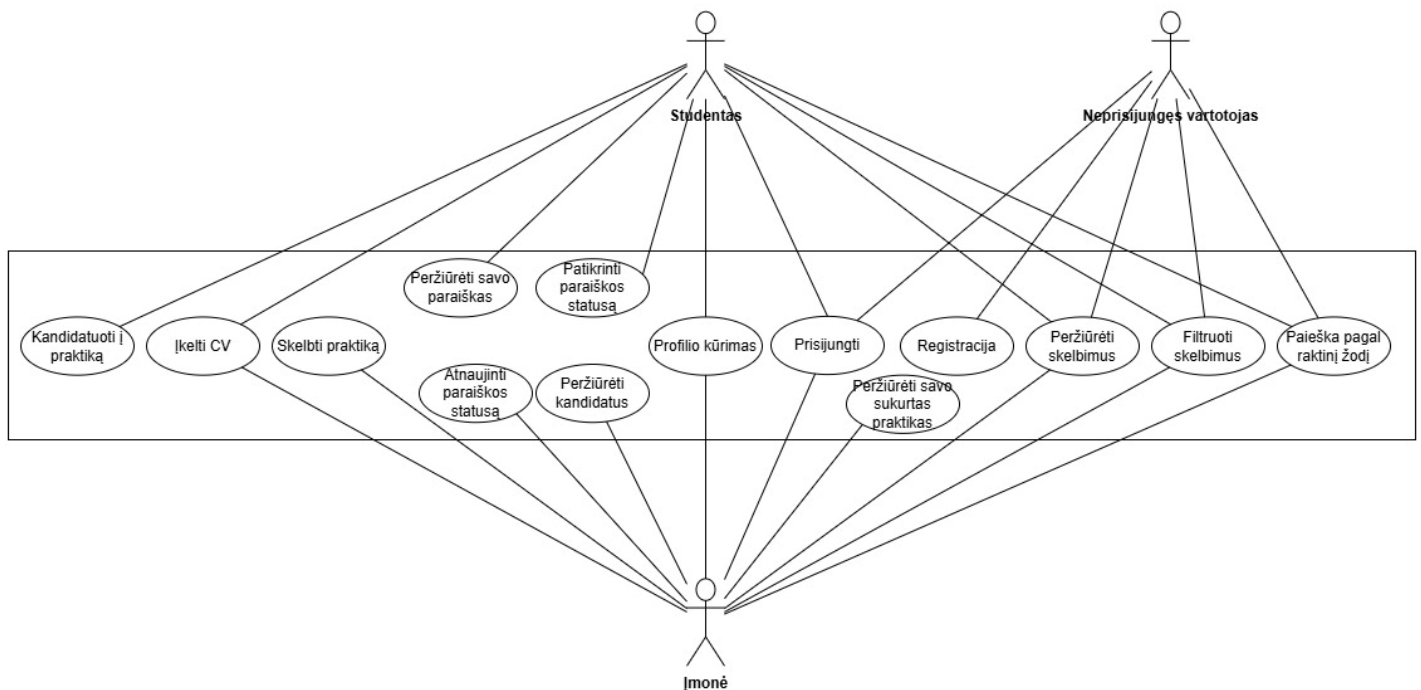
Projekto įgyvendinimui buvo nuspręsta naudoti React biblioteką kliento daliai bei Node.js su Prisma API kūrimui ir darbui su duomenų baze. Viena iš pagrindinių priežasčių pasirinkti šias technologijas yra ta, jog visa sistema bus kuriama naudojant JavaScript, todėl tiek kliento, tiek serverio pusės logika bus lengviau suprantama. React pasirinktas dėl komponentinio požiūrio ir plačios ekosistemos, leidžiančios greitai kurti patogią vartotojo sąsają. Node.js pritaikytas dėl asinchroninio veikimo ir paprasto REST API kūrimo, o Prisma – kaip modernus ORM, užtikrinantis tipais paremtą užklausų rašymą. Duomenų saugojimui pasirinkta PostgreSQL reliacinė duomenų bazė, nes ji pasižymi stabilumu, patikimumu ir geru sudėtingų ryšių tarp lentelių palaikymu, kas būtina kuriant praktikų bei paraiškų valdymo sistemą. Passport Google OAuth autentifikacijai, Multer – failų įkėlimams (CV ir logotipams), Nodemailer – el. laiškų siuntimui. Sistemos diagramoms braižyti naudojamas draw.io, nes jis paprastas ir plačiai naudojamas įrankis.

4. NEFUNKCINIAI REIKALAVIMAI

Kuriamai sistemai keliama keli pagrindiniai nefunkciniai reikalavimai. Našumui padėti naudojamas PostgreSQL ryšių baseinas (pg Pool, backend/db.js) ir skelbimų puslapiavimas (GET /api/praktikos su page/per_page), kas sumažina vienkartinių didelių užklausų apkrovą. Saugumo srityje autentifikacija vykdoma per Google OAuth, serverio pusėje atliekamos rolės patikros prieš jautrias operacijas, SQL užklausos parametrizuotos per pool.query (apsauga nuo SQL injection), o failų įkėlimai turi tipų ir dydžio patikras (CV 2MB, logo 1MB). Suderinamumui užtikrinti frontend yra React + Vite, o API naudoja JSON, todėl klientas ir serveris keičiasi standartiniais formatais ir dirba pagrindinėse naršyklėse.

5. PANAUDOS ATVIEJŲ DIAGRAMA

Studentų ir įmonių praktikos paieškos sistemos panaudos atvejų diagrama



1 pav. Studentų ir įmonių praktikos paieškos sistemos panaudos atvejų diagrama

Panaudos atvejų diagramoje (1 pav.) vaizduojamos trys rolės: studentas, įmonė ir neprisijungęs vartotojas.

Bendros vartotojų funkcijos:

- Paieška pagal raktinį žodį
- Skelbimų filtravimas
- Skelbimų peržiūra
- Prisijungimas

Tik studentui suteiktos funkcijos:

- Kandidatuoti į praktiką
- Įkelti CV
- Peržiūrėti savo paraiškas

- Patikrinti paraiškos statusą
- Profilio kūrimas

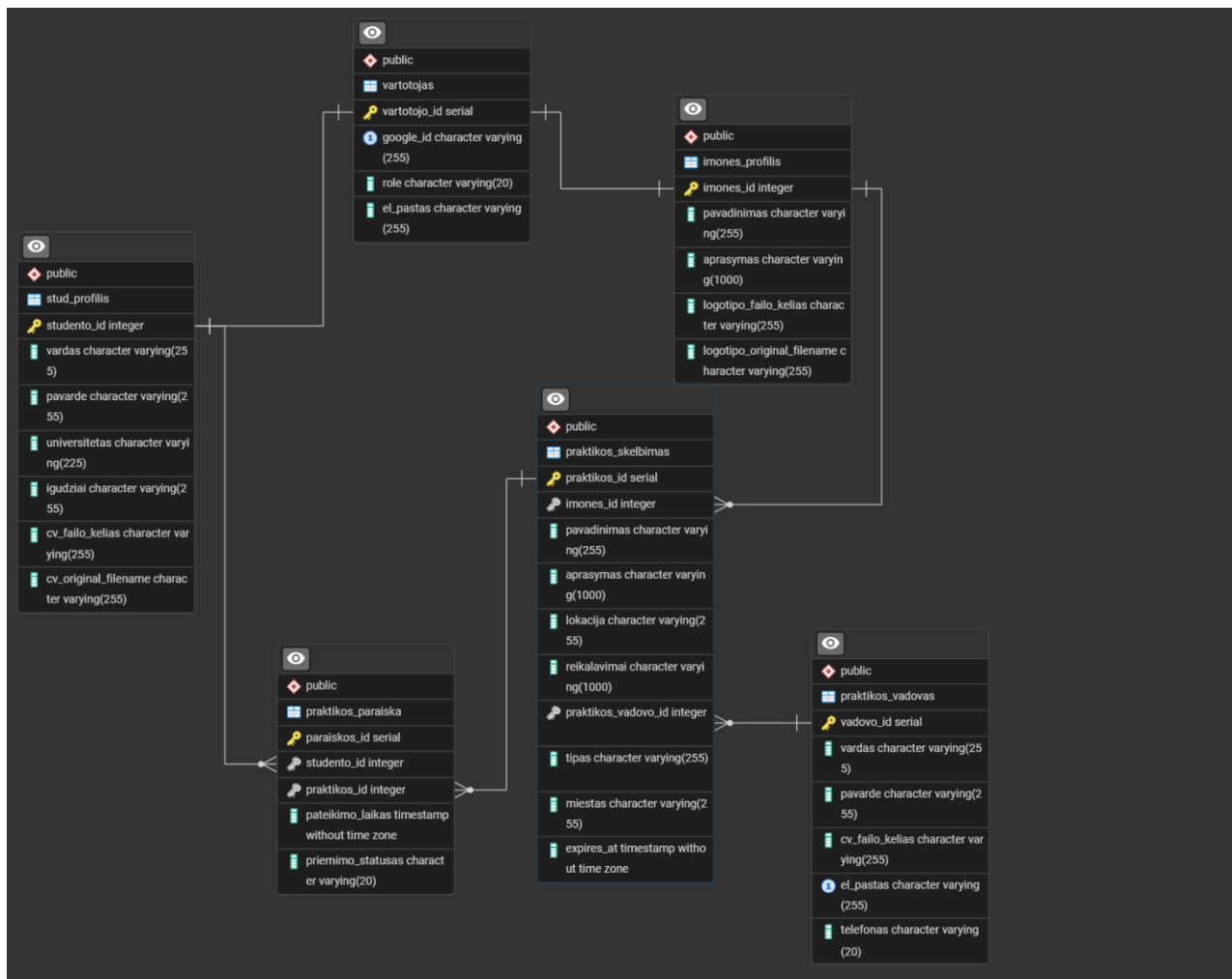
Tik įmonei suteiktos funkcijos:

- Skelbti praktiką
- Atnaujinti paraiškos statusą
- Peržiūrėti kandidatus
- Peržiūrėti savo sukurtas praktikas
- Įkelti praktikos vadovo CV
- Profilio kūrimas

Tik neprisijungusiam vartotojui suteiktos funkcijos:

- Registracija

6. RELIACINĖS DUOMENŲ BAZĖS SCHEMA

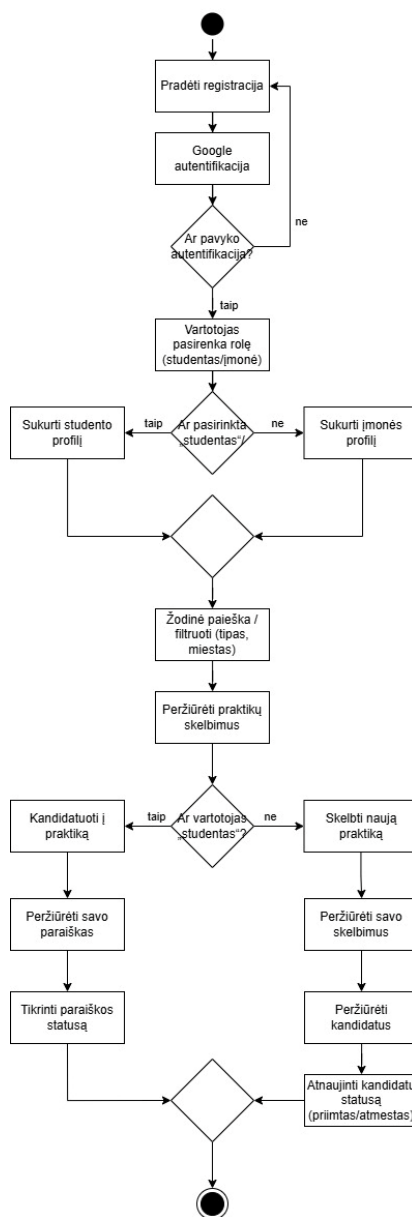


2 pav. Reliacinės duomenų bazės schema

Vartotojų ir duomenų reliacinė schema (2 pav.) vizualiai atspindi duomenų struktūrą. Šioje diagramoje „Vartotojas“ lentelė veikia kaip pagrindinė, sauganti bendrą informaciją, o „Studento profilis“ ir „Įmonės profilis“ lentelės yra susietos „vienas su vienu“ ryšiu, kas leidžia kiekvienam vartotojui turėti tik vieną specializuotą profilį, naudojant tą patį ID kaip pirminį (PK) ir svetimą (FK) raktą. Tuo tarpu „daug su daug“ ryšys tarp „Studento profilis“ ir „Praktikos skelbimai“ yra

išskaidytas į dvi „vienas su daug“ sąsajas per tarpinę lentelę „Praktikos paraiška“, kuri leidžia saugoti paraiškos būseną ir kitus papildomus duomenis. „Praktikos vadovas“ esybė turi ryšį „vienas su daug“ su „Praktikos skelbimas“.

7. AKTYVUMO DIAGRAMA



3 pav. Vartotojų sąveikos aktyvumo diagrama

Vartotojo sąveikos aktyvumo diagrama (3 pav.) vizualizuoja veiksmų seką, kurią atlieka vartotojas, norėdamas užsiregistruoti ir naudotis sistema, atsižvelgiant į jo pasirinktą rolę. Procesas prasideda nuo registracijos ir Google autentifikacijos. Po sėkmingos autentifikacijos sistema leidžia vartotojui pasirinkti, ar jis yra studentas, ar įmonė – šis pasirinkimas skiria veiksmų srautą į dvi skirtingas šakas, kur pagal pasirinktą rolę kuriamas vartotojo profilis. Sukūrus profilį vartotojai gali naudotis žodine paieška, filtravimu (pagal ieškomos srities tipą, miestą), peržiūrėti praktikos skelbimus. Tada vėl skiriamas veiksmų srautas pagal vartotojo rolę. Studentai gali pasirinkti skelbimą ir teikti paraiškas, vėliau peržiūrėti savo paraiškas ir tikrinti paraiškų statusą. Įmonės gali skelbti naujas praktikas, peržiūrėti savo skelbimus ir kandidatus bei atnaujinti

kandidato priėmimo statusą (priimta/atmesta). Ši diagrama padeda suprasti, kaip informacija teka sistemoje ir kaip vartotojai sąveikauja su skirtingais funkcionalumais priklausomai nuo jų rolės.

8. PAKEITIMAI

Darbo eigoje buvo atlikti keitimai, kurie patobulino ar padėjo įgyvendinti kuriamą sistemą.

8.1. Iteracijų pakeitimas

Pradinėje versijoje, pirmoje iteracijoje buvo numatytos funkcijos:

- Prisijungimas
- Registracija (profilio sukūrimas, rolės pasirinkimas: studentas ar įmonė)
- El. pašto patvirtinimas

Darbo eigoje iteracija pakeista į:

- Prisijungimas/registracija su Google auth
- Profilio sukūrimas (rolės pasirinkimas: studentas ar įmonė, kita vartotojo informacija)

8.2. Technologinių sprendimų pakeitimas

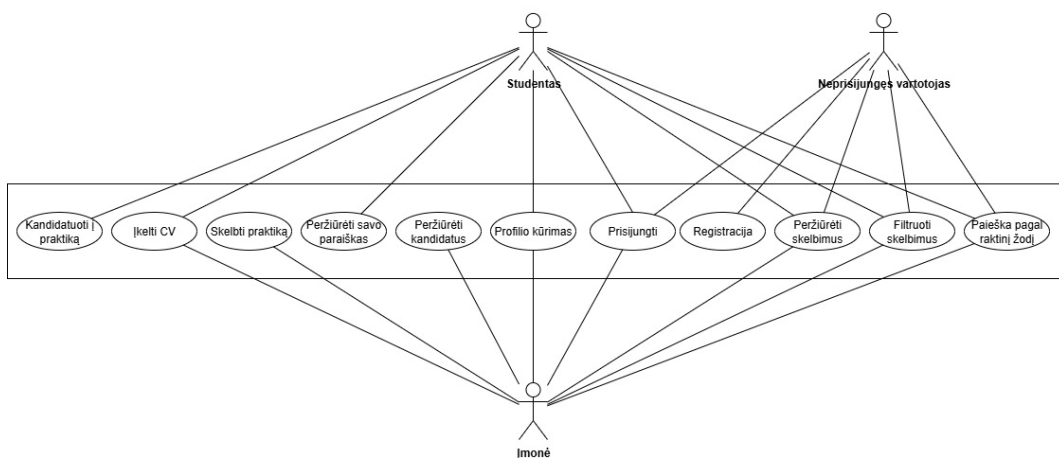
Iš technologijų, kurių pradiniam aprašyme nebuvo paminėta, projekte yra įdiegta Passport Google OAuth autentifikacijai (pakeistos iteracijos funkcijai įgyvendinti), Multer (failų įkėlimams – CV ir logotipams), Nodemailer (el. laiškų siuntimui).

8.3. Panaudos atvejų diagramos pakeitimas

Atidžiau peržiūrėjus I versijos panaudos atvejų diagramą buvo pastebėta, kad trūksta kelių funkcijų tad prie senos versijos (4 pav.) pridėtos trys papildomos funkcijos (5 pav.):

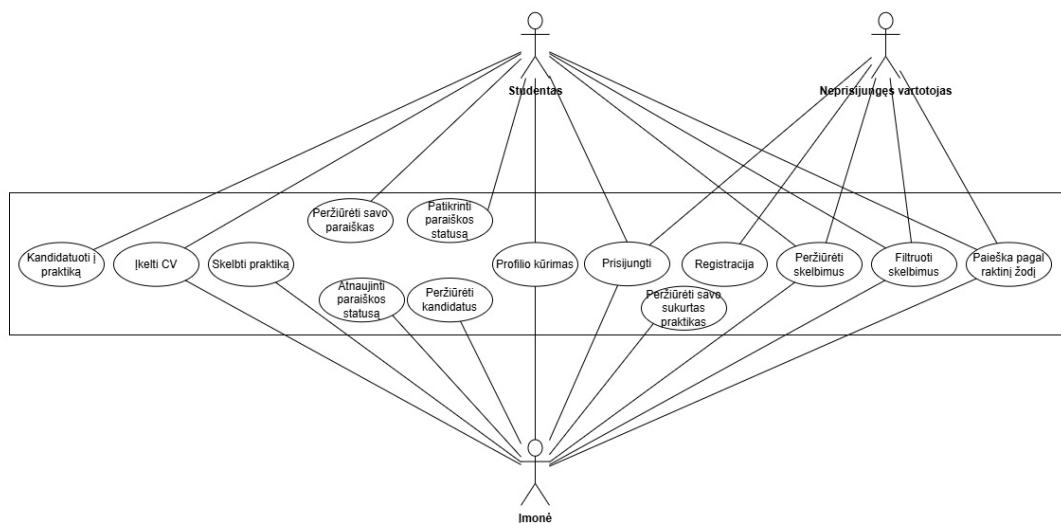
- Patikrinti paraiškos statusą (studentas)
- Atnaujinti paraiškos statusą (įmonė)
- Peržiūrėti savo sukurtas praktikas (įmonė)

Studentų ir įmonių praktikos paieškos sistemos panaudos atvejų diagrama



4 pav. Panaudos atvejų diagrama (I versija)

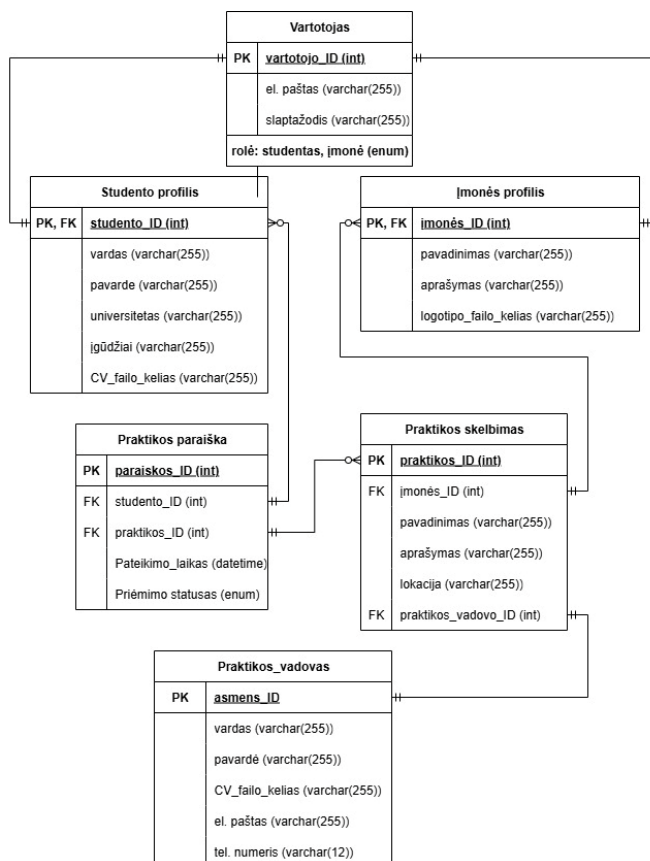
Studentų ir įmonių praktikos paieškos sistemos panaudos atvejų diagrama



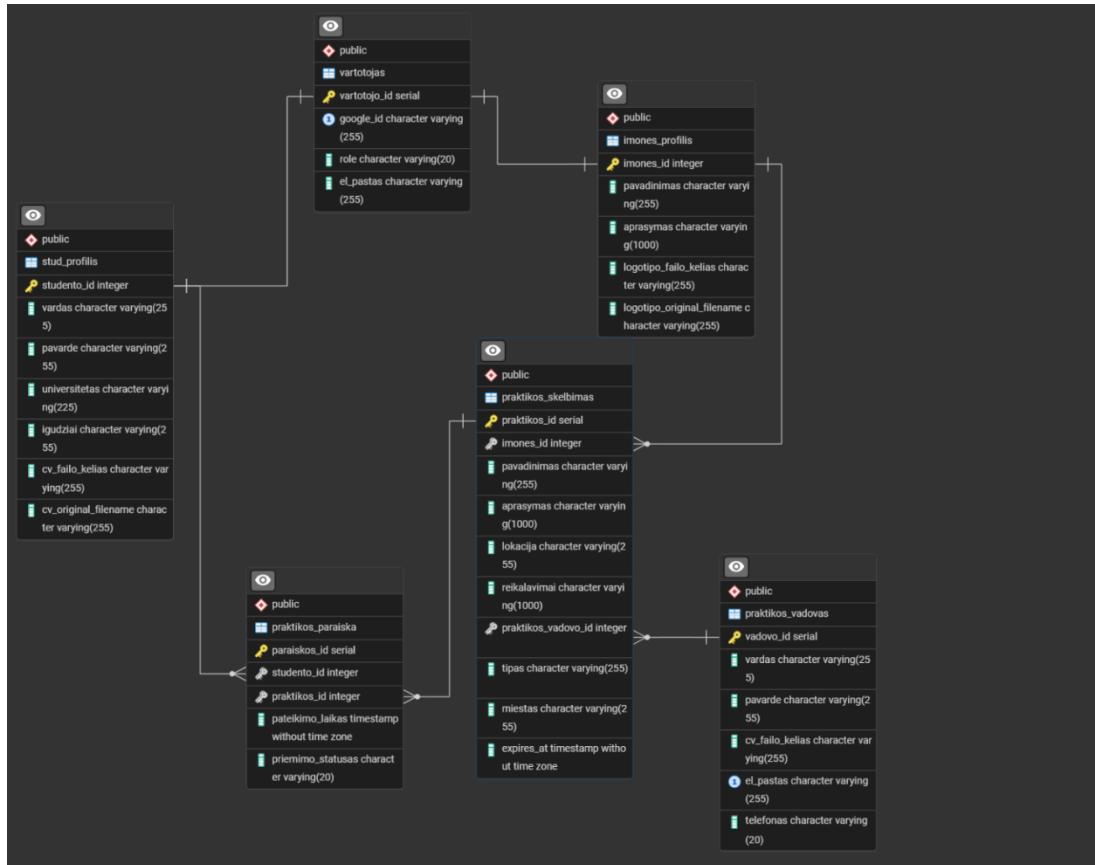
5 pav. Panaudos atvejų diagrama (II versija)

8.4. Esybių ryšių diagramos pakeitimai

Esybių ryšių diagrama vietoje vartotojo slaptažodžio dabar saugo google_id (pereita prie Google OAuth autentifikacijos), todėl laukelis slaptažodis išnyko. Profilio lentelėse atsirado papildomi duomenų laukai: stud_profiles turi cv_original_filename, imones_profilis turi logotipo_original_filename (kad išsaugotų originalius failų pavadinimus). Praktikos skelbimo lentelė išsiplėtė su laukais, reikalingais paieškai: pridėtas tipas, miestas, reikalavimai ir expires_at (galiojimo data). Taip pat padidinti laukelio aprašymams ilgis iš 255 į 500.



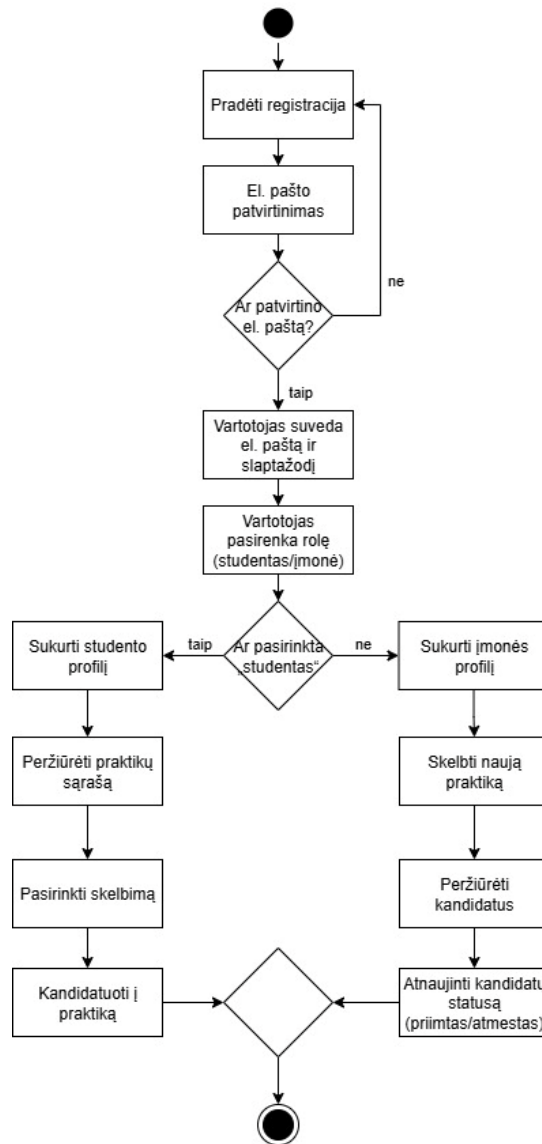
6 pav. Reliacinės duomenų bazės schema (I versija)



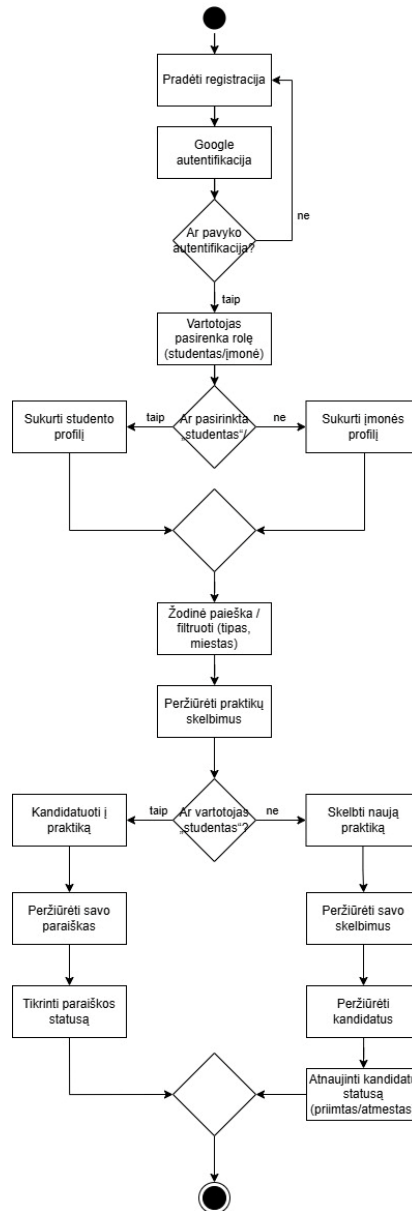
7 pav. Reliacinės duomenų bazės schema (II versija)

8.5. Aktyvumo diagramos pakeitimai

Senoje diagramoje (8 pav.) el. pašto patvirtinimas, vartotojo el. pašto/slaptažodžio suvedimas pakeisti į prisijungimą per Google (9 pav.). Taip pat pakeistas srautas: po profilio kūrimo, abi rolės peradresuojamos į bendrą srautą, kur gali atlikti žodinę paiešką, filtravimą ir skelbimų peržiūrą. Tolimesnis srautų išskaidymas atliekamas kaip ir senosioje versijoje.



8 pav. Vartotojų sąveikos aktyvumo diagrama (I versija)



9 pav. Vartotojų sąveikos aktyvumo diagrama (II versija)

9. PALEIDIMO INSTRUKCIJA

Prieš pradedant, įsitikinti, kad yra įdiegta:

- Node.js (LTS versija, pvz. 18.x arba 20.x)
- PostgreSQL (14 ar naujesnė)
- Npm (paprastai įdiegiamas su Node.js)
- Git (repositorijai klonuoti)

Atidarykite terminalą

9.1. Repositorijos klonavimas

Į terminalą įveskite:

- `git clone https://github.com/EligMaa/studentu-imoniu.git`
- `cd studentu-imoniu`

Patikrinkite versijas:

- node -v (turėtų būti v18.x ar naujesnė)
- npm -v (turėtų būti v9.x ar naujesnė)
- npm install

9.2. Duomenų bazės sukūrimas

WINDOWS (PowerShell):

- psql -U postgres
- CREATE USER appuser WITH PASSWORD 'jusu_saugus_slaptazodis';
- CREATE DATABASE praktika OWNER appuser;
- GRANT ALL PRIVILEGES ON DATABASE praktika TO appuser;
- \q

LINUX/MACOS:

- sudo -u postgres psql
- CREATE USER appuser WITH PASSWORD 'jusu_saugus_slaptazodis';
- CREATE DATABASE praktika OWNER appuser;
- GRANT ALL PRIVILEGES ON DATABASE praktika TO appuser;
- \q

Įkelkite bazinę schemą:

- cd studentu-imoniu/backend
- psql -U appuser -h localhost -d praktika -f database.sql

Įvykdyskite migracijas (chronologine tvarka):

- cd db_migrations
- psql -U appuser -h localhost -d praktika -f add_praktikos_tipas.sql
- psql -U appuser -h localhost -d praktika -f add_praktikos_miestas.sql
- psql -U appuser -h localhost -d praktika -f add_praktikos_expires.sql
- psql -U appuser -h localhost -d praktika -f complete_schema_migration.sql

Patikrinkite lentelių sąrašą:

- psql -U appuser -h localhost -d praktika -c "\dt"

9.3. Google OAuth konfigūracija

1. Eikite į <https://console.cloud.google.com/>
2. Sukurkite naują projektą
3. Įjunkite "Google+ API"
4. Credentials → Create Credentials → OAuth 2.0 Client ID
5. Application type: Web application
6. Authorized redirect URIs: <http://localhost:3000/auth/google/callback>

7. Nukopijuokite CLIENT_ID ir CLIENT_SECRET

9.4. Environment variables konfigūracija

Backend (.env failas backend/ kataloge):

PORT=4000

COOKIE_SECRET= sugeneruokite_random_string_32_chars

CLIENT_URL=http://localhost:5173

NODE_ENV="development"

DATABASE_NAME=praktika

DATABASE_HOST=localhost

DATABASE_USER=appuser

DATABASE_PASSWORD=jusu_saugus_slaptazodis

CLIENT_ID=your_google_client_id_here

CLIENT_SECRET=your_google_client_secret_here

GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

EMAIL_USER=internlink0@gmail.com

EMAIL_PASSWORD=czsfbzsphkdsfxst

9.5. Dependencies įdiegimas

Backend:

- cd backend
- npm install

Frontend (naujame terminale):

- cd frontend
- npm install

9.6. Paleidimas

BACKEND (terminalas 1):

- cd backend
- npm start

FRONTEND (terminalas 2):

- cd frontend

- npm run dev

Atidarykite naršyklėje <http://localhost:5173>

10. PROJEKTO STRUKTŪRA

Rašant kodą buvo pasiremta šaltiniais:

- Google autentifikacijai (auth.js, authRouter.js, index.js):
https://www.youtube.com/watch?v=2SEz6SK_ekE
- Migracijų valdymui, pg bibliotekų naudojimui, duomenų bazės prijungimui, duomenų validacijai ir saugumui (database.sql, db_migrations/*.sql, run_migration.js, db.js):
<https://www.youtube.com/watch?v=o9e3ex-axzA>
- Passport.js autentifikacijos pagrindai, sesijomis grįsta autentifikacija, maršrutų apsauga (auth.js, index.js, isAuth.js, authRouter.js):
[Express JS #15 - Passport.js Authentication](https://www.youtube.com/watch?v=2SEz6SK_ekE)
- Cors biblioteka (index.js):
<https://github.com/expressjs/cors>
- Pg biblioteka (db.js, praktikosRouter.js, profileRouter.js, index.js):
<https://node-postgres.com/apis/pool>
- Dotsenv biblioteka (index.js, db.js, run_migration.js, .env):
<https://github.com/motdotla/dotenv>
- Multer biblioteka (profileRouter.js, praktikosRouter.js):
<https://github.com/expressjs/multer>
- Nodemailer biblioteka (emailService.js, praktikosRouter.js):
<https://nodemailer.com/>
- AI įrankis buvo naudojamas try-catch blokams su detaliomis console.log žinutėmis, formų validacijoms su field-level error handling, react komponentui skeleton loading. Visas sugeneruotas kodas buvo peržiūrėtas, adaptuotas projektui, patikrintas.